

Hurricane Creek Research Site Application

Wyatt A. Straksis
University of North Georgia
May 1, 2026

Abstract

This report covers the design and development of an application for the Environmental Sciences Department at UNG that allows them to capture, organize, and retrieve geo-tagged photos with their metadata attached. This was requested by the Environmental Leadership Center (ELC) due to their desire to have a main hub to view data collected within the Hurricane Creek Research Site for ease of access during their individual research projects within the team. To implement this application, multiple technologies were used: Postgres, React, Node.js, and Express for the building of the application. Additionally, Vercel, Railway, and Cloudinary were utilized for the deployment of the application. The results of this application were well-received among the eight users that tested it with many giving the max rating on a survey question that asked “how likely would you be to recommend this application to peers or faculty members?” A few issues regarding upload failure did occur and were resolved quickly to provide a stable final product.

Keywords: Progressive Web App, Full-Stack, React, Node.js, PostgreSQL, Cloudinary, Environmental Research

1. Introduction

The ELC is an organization that promotes service, research, and leadership in the field of preserving and enhancing the environment. It consists of students, faculty, and community members who work together to create a noticeable change within the North Georgia region by utilizing the University of North Georgia's (UNG) Environmental Sciences Department as its base of operations. Its primary research ground is known as Hurricane Creek Research Site, a seventy-two acre plot of land residing in Dahlonega, Georgia.

Before my application, the ELC utilized data through various mediums for their research: data held by faculty alone that isn't shared among others, data in existing databases, and data on paper. This was a major issue for their research workflow because they had to constantly juggle workflow with collecting information from a diverse variety of sources, some of which they couldn't even fully access the information from. This decreased the overall productivity and the accuracy of their research, which meant it was time for a change.

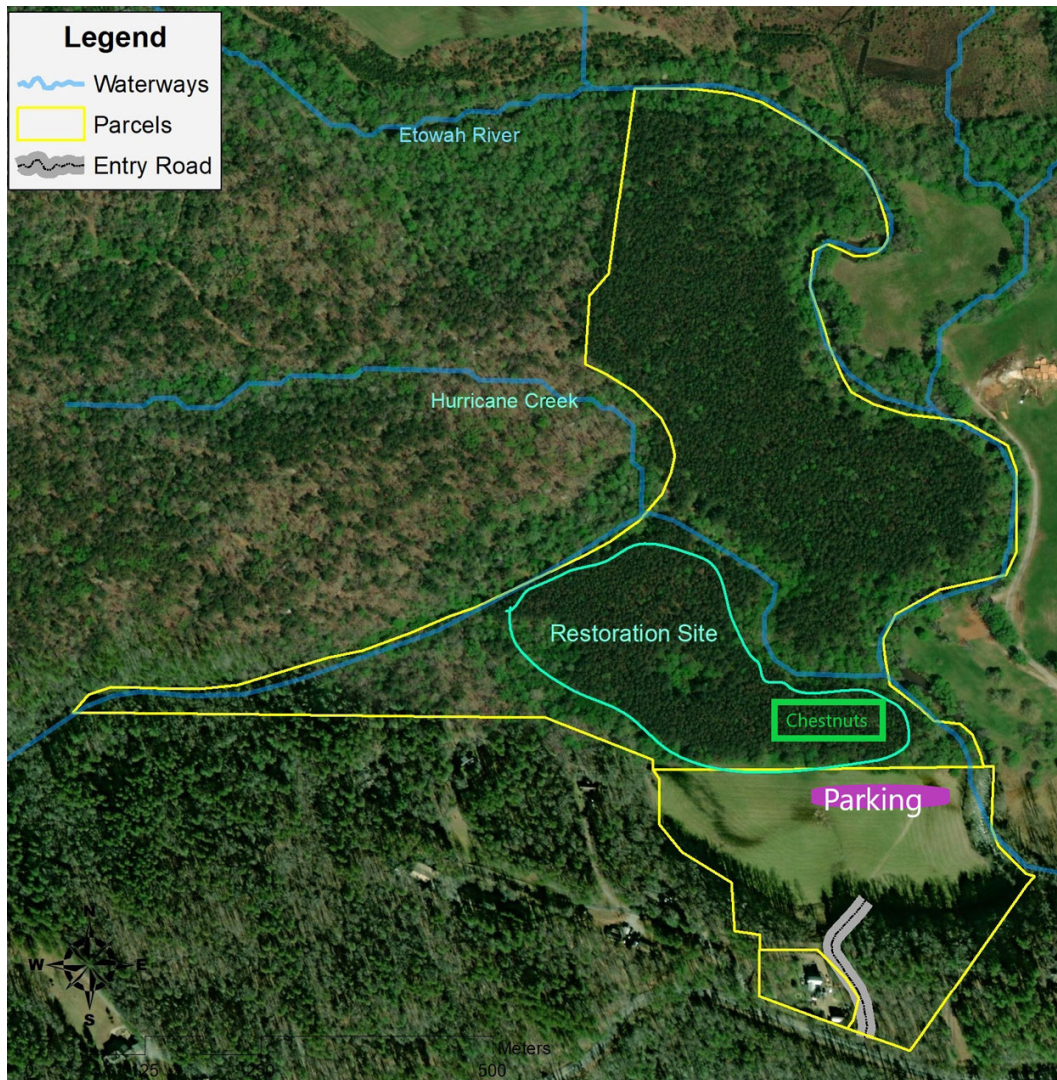


Figure 1: Map of the Hurricane Creek Research Site utilized by the ELC.

This is when I was contacted regarding the development of an application meant to serve as a hub for all their research information. An application meant to standardize the data collection and submission process among faculty, students, and volunteers. The resulting application standardizes this process so that all users upload and attach data in the same format, and all faculty members can access it equally.

Throughout the rest of this document, I will detail how I built this application, the issues that arose and how I solved them, alongside the impact it had on the Environmental Sciences Department as a whole.

2. Description of Solutions

The application was designed as a Full-Stack Progressive Web App (PWA). These two aspects warrant clarification: the meaning of Full-Stack and the meaning of PWA. A Full-Stack application can be defined as an application that utilizes front-end, back-end, and database architecture working in tandem to create a self-sufficient platform. On the other hand, a PWA is a website that additionally has the capacity to function as a native application. This means that it can be added to a user's home screen on their phone and act as an app. This report addresses the following areas of the application's architecture: Frontend, Backend, Database, Image Handling, and Deployment.

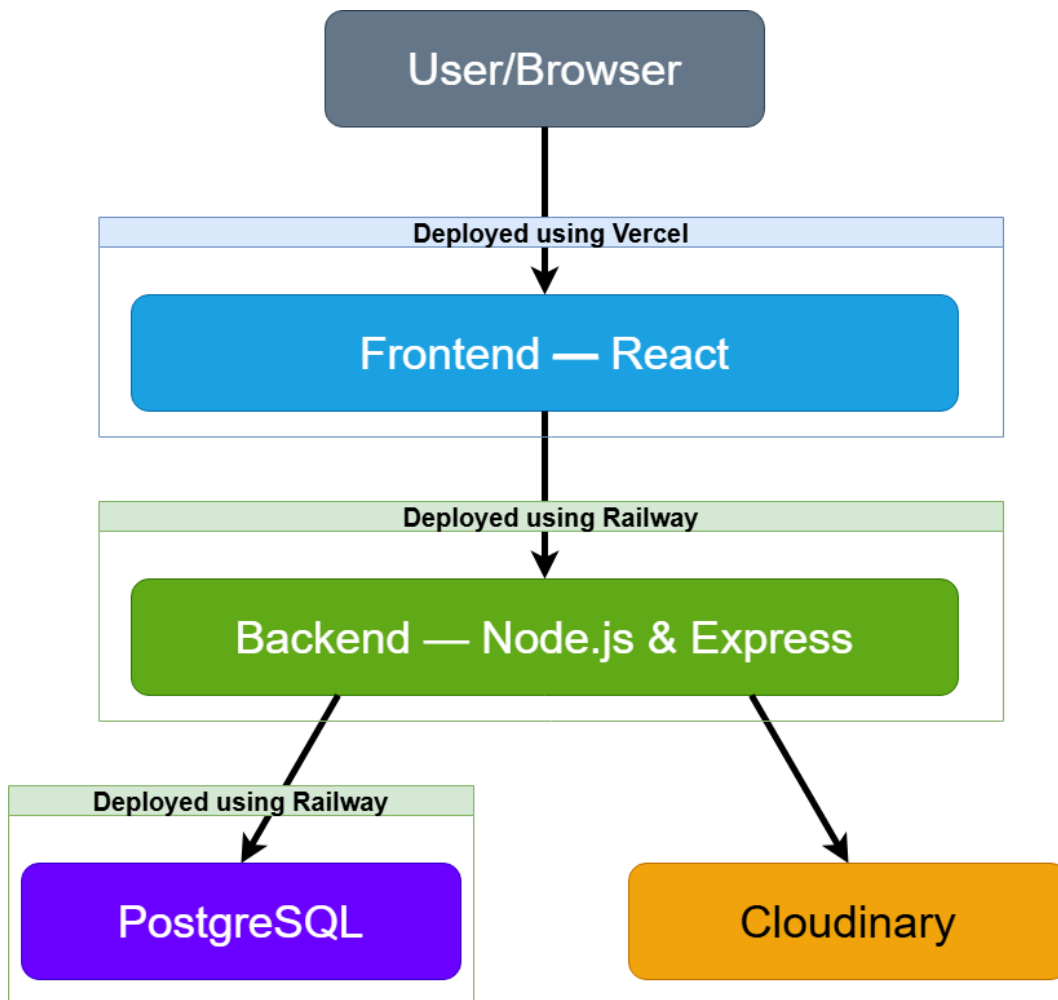


Figure 2: System architecture diagram showing the relationship between the Frontend, Backend, Database, and cloud services.

2.1. Frontend

The Frontend was built with React [1] and acts as the intermediary between user input and the backend. The main way I accomplished this was utilizing `useState` and `useEffect` to manage application state and handle asynchronous API calls to the backend. When a user interacts with UI elements or uploads information, all of that triggers different request methods within the system architecture that connect to the backend to either retrieve or send information. For example, when a user uploads a photo on the upload page: that photo upload calls several request methods that change the UI of the page to display the newly uploaded photo and also prepare the photo to be converted and uploaded when the user eventually finishes the uploading process.

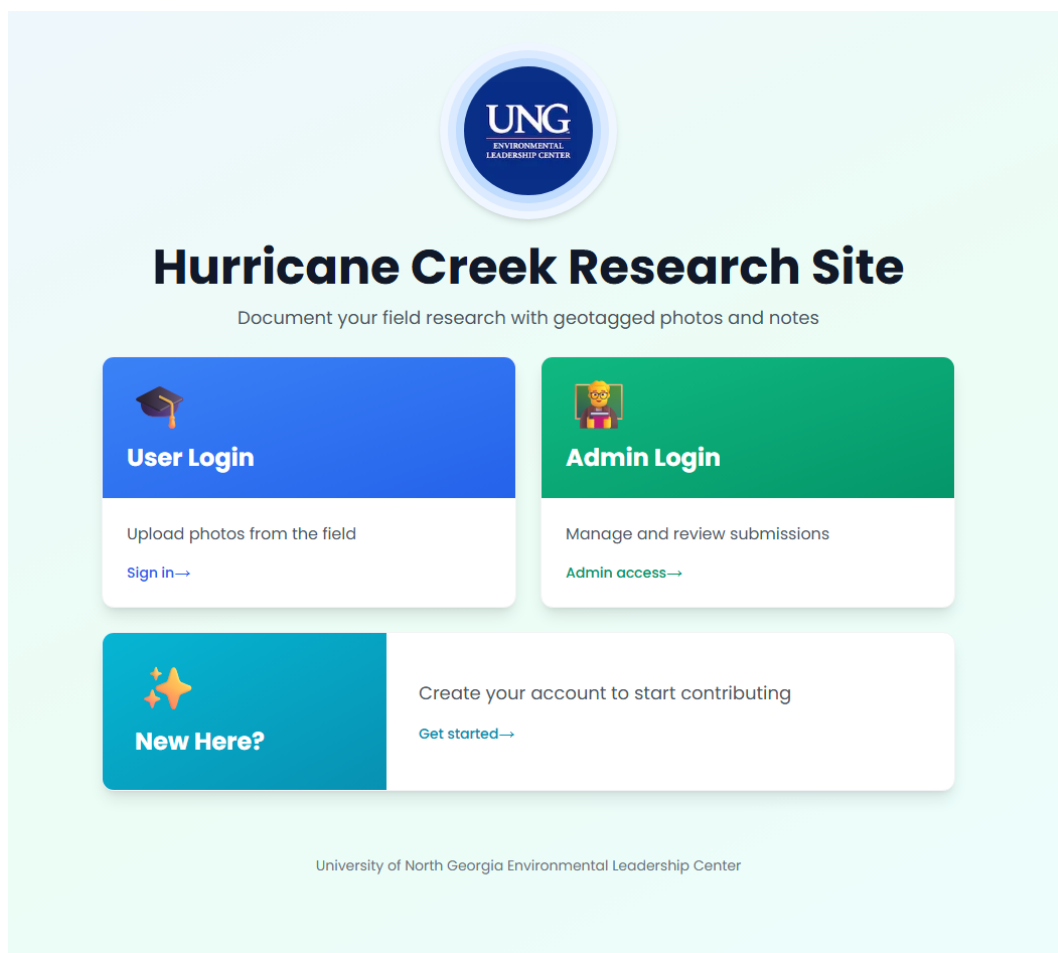


Figure 3: The application login screen, the entry point for all users.

2.2. Backend

The Backend was built with Node.js [2] and Express [3]: its primary function is to act as the intermediary between the front-end and the database. Additionally, it houses the calls and logic for Cloudinary, which will be discussed further in the Image Handling subsection. The backend is mainly composed of POST and GET middleware to handle transactions between different components of the application’s system. Routes are organized by resource, with each route handler focusing on a specific component, such as photo uploads, zone management, or administrative functions.

2.3. Database

The Database utilizes PostgreSQL [4] and acts as the storage for user data, Cloudinary URLs, and uploaded information. There are four tables within the database: users, zones, photos, and audit logs. The users table handles all user information that the application needs to function, such as their email, role, name, and password. The password in this database is hashed in order to securely store it. When a user enters their password, the password they enter is hashed and compared against this stored hashed password and allows access if they are the same. The zones table handles all zone information, which is the name and id of the different zones users can select while uploading. This serves to allow for ease of filtration while viewing images in the gallery as an admin. The photos table stores foreign keys from the users and zone tables, alongside necessary metadata that may be uploaded alongside photos, such as latitude, longitude, and notes. Finally, the audit logs store information about the users who have uploaded photos, specifically the uploader’s name and timestamp, providing an auditable record of all photo submissions.

Table 1 summarizes the four database tables and their primary fields.

Table 1: Database Schema Overview	
Table	Primary Fields
users	id, email, name, password_hash, role, created_at
zones	id, name
photos	id, user_id, zone_id, category, notes, latitude, longitude, image_url, status, created_at
audit_logs	id, actor_user_id, target_photo_id, action, detail, created_at

2.4. Image Handling

Image Handling is done entirely through Cloudinary [5], a cloud-based media management platform that enables storage and control over the photos uploaded to it. There are two primary reasons for this external software being utilized. The first is that storing images directly in the database would result in a massive increase of storage usage and lead to numerous problems with the application in the future once more photos are uploaded. Cloudinary solves this by allowing photos to be uploaded to its own cloud storage and providing a URL which can then be stored in the application's database and used to retrieve the photo at any time. Second, Cloudinary allows a photo to transform image formats during upload. This is useful because any image file can be transformed into a WebP, a modern image format that significantly reduces file size while maintaining visual quality.

2.5. Deployment

Deployment for the application is done utilizing two different platforms: Vercel [6] and Railway [7]. Both of these are cloud-based platforms, but Vercel serves as a host for the front-end of applications and Railway serves as a host for the back-end and database. Both can be used independently but function well together, and using both of these platforms has allowed the application to be hosted entirely online and remain active at all times. Additionally, since these platforms connect to the GitHub branches that they are pulling the code from, any time that the GitHub branch is updated, the platforms will automatically redeploy.

3. Results and Discussion

Once the application was deployed, field testing could begin. Testing was conducted on a user-oriented basis, meaning that users could choose when and how much they tested with the application. The goal behind this was to have them integrate the application into their normal routines and effectively simulate what department-wide usage of this application may be like. This was very effective, with around eight users signing up by the end of the two-week testing phase and around thirty photos being added to the database.

Photo *



Click to select photo
or drag and drop

Zone/Habitat *

Select a zone... ▼

 **View Site Map**

Category *

Select a category... ▼

Observation Notes

Describe what you observed... (optional)

GPS Status

● Location services disabled

Enable GPS in your dashboard to include location data

Upload to Research Site

Figure 4: The upload interface presented to field researchers during testing.

To further garner information, I sent out surveys to both users and admins to receive feedback on the application and the upload process as a whole. One user described the upload process as “remarkably intuitive, but a good way to gather quite a bit of data.” The general consensus seemed to be that the user side of the application was well-received. On the other hand, an admin provided detailed feedback, suggesting the addition of editorial controls and observer-based sorting for future versions. This is a fair statement and I agree

with all points they provided. In fact, most of their ideas were concepts that I already had planned for version two.

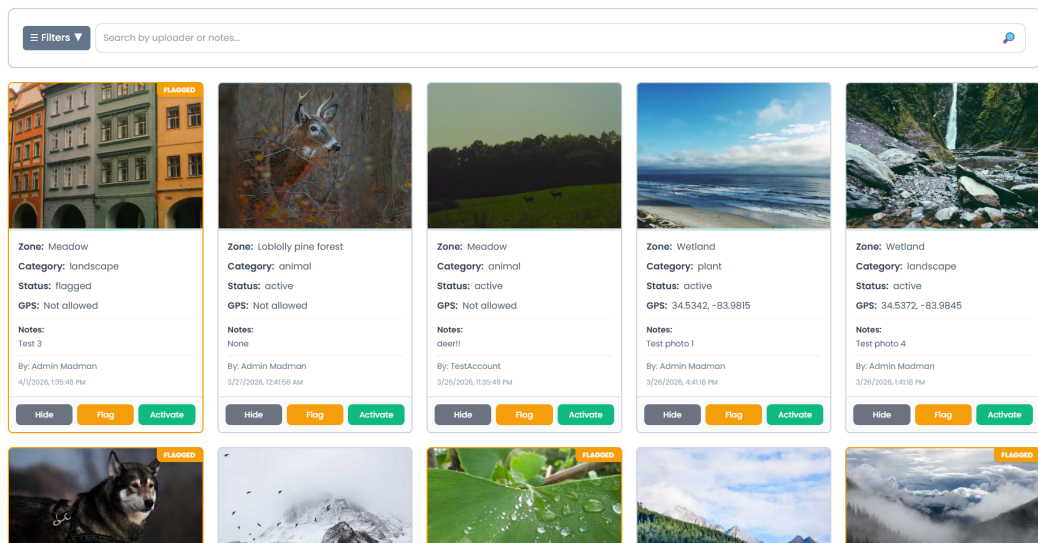


Figure 5: The admin gallery view displaying uploaded photos with filtering and pagination controls.

However, not everything was without issue in the beginning. One of the major issues that arose was that photo uploads were taking a long time and sometimes causing the upload process to fail entirely. Upon investigation, I discovered that the issue was file size, and there was a simple solution: change the file type that is stored. PNG and JPEG file types can be large, but they can be reduced in size by transforming them into the aforementioned WebP file type. Upon instituting this change, there was a significant improvement in upload speed and nearly a ninety-five percent reduction in file size.

 tegan-conway-loCow_3_DQI-unsplash.jpg File Size: 19685.8 KB Dimensions: 5152x7728px	 ReplacedNameJustinCase.webp File Size: 1034.1 KB Dimensions: 1365x2048px																		
File Information <table border="1"> <tr><td>File Size</td><td>20,158,264 Bytes / 19685.8KB / 19.22MB</td></tr> <tr><td>File Type</td><td>JPEG</td></tr> <tr><td>File Type Extension</td><td>JPG</td></tr> </table>	File Size	20,158,264 Bytes / 19685.8KB / 19.22MB	File Type	JPEG	File Type Extension	JPG	File Information <table border="1"> <tr><td>File Size</td><td>1,058,904 Bytes / 1034.1KB / 1.01MB</td></tr> <tr><td>File Type</td><td>Extended WEBP</td></tr> <tr><td>File Type Extension</td><td>WEBP</td></tr> </table>	File Size	1,058,904 Bytes / 1034.1KB / 1.01MB	File Type	Extended WEBP	File Type Extension	WEBP						
File Size	20,158,264 Bytes / 19685.8KB / 19.22MB																		
File Type	JPEG																		
File Type Extension	JPG																		
File Size	1,058,904 Bytes / 1034.1KB / 1.01MB																		
File Type	Extended WEBP																		
File Type Extension	WEBP																		
Image Properties <table border="1"> <tr><td>Aspect Ratio</td><td>2 / 3</td></tr> <tr><td>ResolutionUnit</td><td>1</td></tr> <tr><td>X Resolution</td><td>72</td></tr> <tr><td>Y Resolution</td><td>72</td></tr> <tr><td>ColorSpaceData</td><td>RGB</td></tr> </table>	Aspect Ratio	2 / 3	ResolutionUnit	1	X Resolution	72	Y Resolution	72	ColorSpaceData	RGB	Image Properties <table border="1"> <tr><td>Aspect Ratio</td><td>2 / 3</td></tr> <tr><td>Image Width</td><td>1365</td></tr> <tr><td>Image Height</td><td>2048</td></tr> <tr><td>ColorSpaceData</td><td>RGB</td></tr> </table>	Aspect Ratio	2 / 3	Image Width	1365	Image Height	2048	ColorSpaceData	RGB
Aspect Ratio	2 / 3																		
ResolutionUnit	1																		
X Resolution	72																		
Y Resolution	72																		
ColorSpaceData	RGB																		
Aspect Ratio	2 / 3																		
Image Width	1365																		
Image Height	2048																		
ColorSpaceData	RGB																		

Figure 6: Comparison of file sizes before and after WebP conversion, demonstrating the reduction in storage usage.

Another issue that arose was security of the application. Without proper implementation, my application could be vulnerable to malicious parties who want to extract information like user passwords or to skip user authentication entirely. To combat this, I implemented basic security measures, such as user authentication and password hashing. The user authentication is a system that does two things: checks if a user is logged in and what role their account is labelled as. If a user is not logged in and attempts to route to a page like the dashboard or photo upload, then they are sent back to the login page without ever being able to view the page they were trying to access. Moreover, if a user attempts to route themselves to an admin-only page, then they will be sent back if they fail to authenticate as an admin role. These may seem like simple additions, but they were not something I initially had in mind when beginning this project.

4. Conclusion

Overall, this application was built specifically to fill a need presented by the ELC and Environmental Sciences Department, and I believe that it meets all aspects of their original requirements. Work on this application will continue to incorporate desired features and to improve upon existing ones; however, in terms of the first version requirements, everything has been met.

I learned quite a bit from this project, including: full-stack app development, deployment over cloud-based platforms, working with a stakeholder on a real-world project, and how to keep a schedule that keeps a large project on track. I believe that all of these skills will serve me well into the future and are necessary for nearly any profession within the field of

computer science and even outside of it. I would like to thank the ELC for the opportunity to work on a meaningful real-world project, and I am grateful for all that I have learned as a result of building it.

AI Usage

Claude by Anthropic [8] was utilized as an AI assistant throughout the development of this project, including debugging assistance, concept clarification regarding full-stack architecture, deployment, and server-side pagination logic.

References

- [1] React, “React Documentation,” Meta Open Source, 2024. [Online]. Available: <https://react.dev>
- [2] Node.js Foundation, “Node.js Documentation,” 2024. [Online]. Available: <https://nodejs.org/en/docs>
- [3] StrongLoop/IBM, “Express.js Documentation,” 2024. [Online]. Available: <https://expressjs.com>
- [4] PostgreSQL Global Development Group, “PostgreSQL Documentation,” 2024. [Online]. Available: <https://www.postgresql.org/docs>
- [5] Cloudinary Ltd., “Cloudinary Documentation,” 2024. [Online]. Available: <https://cloudinary.com/documentation>
- [6] Vercel Inc., “Vercel Documentation,” 2024. [Online]. Available: <https://vercel.com/docs>
- [7] Railway Corp., “Railway Documentation,” 2024. [Online]. Available: <https://docs.railway.app>
- [8] Anthropic, “Claude,” 2025. [Online]. Available: <https://claude.ai>